

BTS Services Informatiques aux Organisations (SIO)

Session 2026

Nom du candidat : SEILLER

Prénom du candidat : Julian

Parcours : SLAM

Fiche de réalisation professionnelle N°7

Sécurisation applicative (RBAC) et traçabilité des données

d'audit

(Projet CardiacTS)

Période de réalisation : Du lundi 2 mars au vendredi 3 avril

Type de réalisation : En stage (Laboratoire TIMC)

Mode de réalisation : En équipe (Autre stagiaire)

Sommaire

Contexte Organisationnel.....	3
Problématique de gestion.....	3
Choix de la solution mise en œuvre.....	3
Réalisation de la solution.....	4
a - Prérequis et ressources à disposition.....	4
b - Mise en œuvre de la solution.....	4
c - Tests.....	4
i - Cas d'utilisation 1 : Authentification et Hachage.....	4
ii - Cas d'utilisation 2 : Restriction d'accès (RBAC).....	5
iii - Cas d'utilisation 3 : Génération et vérification d'un rapport d'audit.....	5
Conclusions.....	5
Annexes.....	5
Compétences associées.....	6

Contexte Organisationnel

Cette mission s'inscrit toujours dans le cadre de mon stage au **Laboratoire TIMC** (équipe de recherche **BCM**). Dans le développement de l'application de recherche médicale **CardiacTS**, la manipulation d'imageries et de dossiers cliniques imposait des normes strictes de confidentialité. Nôtre rôle en **binôme** a été de concevoir et d'intégrer une couche de sécurité complète pour remplacer l'ancienne gestion "ouverte" sous Excel.

Problématique de gestion

Le laboratoire traite des **données sensibles (médicales)**. L'ancienne méthode (fichiers Excel partagés) ne comportait **aucun contrôle d'accès ni aucune traçabilité**. Les risques étaient multiples :

- Un stagiaire ou un intervenant externe pouvait modifier ou supprimer une donnée clinique par erreur, faussant ainsi les résultats de recherche.
- En cas de modification d'un dossier, il était impossible de savoir *qui* avait effectué le changement et *quand*. Il était impératif de mettre en place un système d'authentification robuste, de restreindre les actions selon le profil de l'utilisateur, et de conserver un **historique inaltérable (Audit)** de toutes les actions pour répondre aux exigences éthiques de la recherche.

Choix de la solution mise en œuvre

Pour sécuriser CardiacTS, nous avons implémenté plusieurs concepts clés de la cybersécurité :

- **Authentification forte** : Utilisation de la bibliothèque **jBCrypt** pour hacher et saler les mots de passe avant leur stockage, évitant toute fuite en cas de compromission de la base de données.
- **Architecture RBAC (Role-Based Access Control)** : Création de profils stricts (**ADMIN** et **USER**). L'interface JavaFX s'adapte dynamiquement pour masquer ou bloquer les fonctionnalités critiques (comme la suppression de dossiers) aux utilisateurs standards.
- **Traçabilité (Audit Trail)** : Création d'un service d'audit métier lié à **PostgreSQL**. Chaque action (connexion, ajout, modification) génère

une entrée automatique dans une table d'historique, permettant la **génération de rapports d'audit**.

- **Protection du code** : Utilisation de fichiers **.env** via la librairie `java-dotenv` pour ne jamais *hardcoder* (écrire en clair) les mots de passe de la base de données dans le code source hébergé sur **GitHub**.

Réalisation de la solution

a - Prérequis et ressources à disposition

- IDE Java (IntelliJ) et framework **JavaFX**.
- Base de données **PostgreSQL**.
- Dépendances Maven : « `jbCrypt` » (sécurité cryptographique) et « `java-dotenv` » (variables d'environnement).

b - Mise en œuvre de la solution

- **Le Portail Sécurisé** : J'ai développé la vue de connexion (`Login.fxml`). Lorsqu'un utilisateur valide le formulaire, le contrôleur récupère le *hash* en base et utilise la méthode `Bcrypt.checkpw()` pour valider l'accès.
- **Gestion des sessions (RBAC)** : J'ai créé une classe **Singleton** « `SessionManager` » qui stocke les informations de l'utilisateur actif. Dans les contrôleurs des autres vues, des conditions vérifient `SessionManager.getRole()`. Si le rôle est `USER`, les boutons "Administration" et "Supprimer" sont désactivés (grisés).
- **Service d'Audit** : J'ai développé un `AuditRepository`. À chaque fois qu'une méthode CRUD d'insertion ou de mise à jour est appelée dans l'application, une requête SQL secondaire est exécutée pour insérer une ligne dans la table `audit_log`(Timestamp, Action, ID Utilisateur, ID Dossier).

c - Tests

i - Cas d'utilisation 1 : Authentification et Hachage

- **Entrée / Action** : Saisie d'identifiants valides dans le **Portail Sécurisé**.
- **Résultat attendu** : La vérification **BCrypt** réussit. Le `SessionManager` est initialisé avec l'identité de l'utilisateur et l'interface principale se charge.

ii - Cas d'utilisation 2 : Restriction d'accès (RBAC)

- **Entrée / Action** : Un chercheur connecté avec un compte de niveau **USER** navigue sur la fiche d'un patient et tente de cliquer sur "Supprimer le dossier".R
- **Résultat attendu** : Le bouton est **verrouillé et invisible**. L'accès au panneau de gestion des utilisateurs lui est également interdit.

iii - Cas d'utilisation 3 : Génération et vérification d'un rapport d'audit

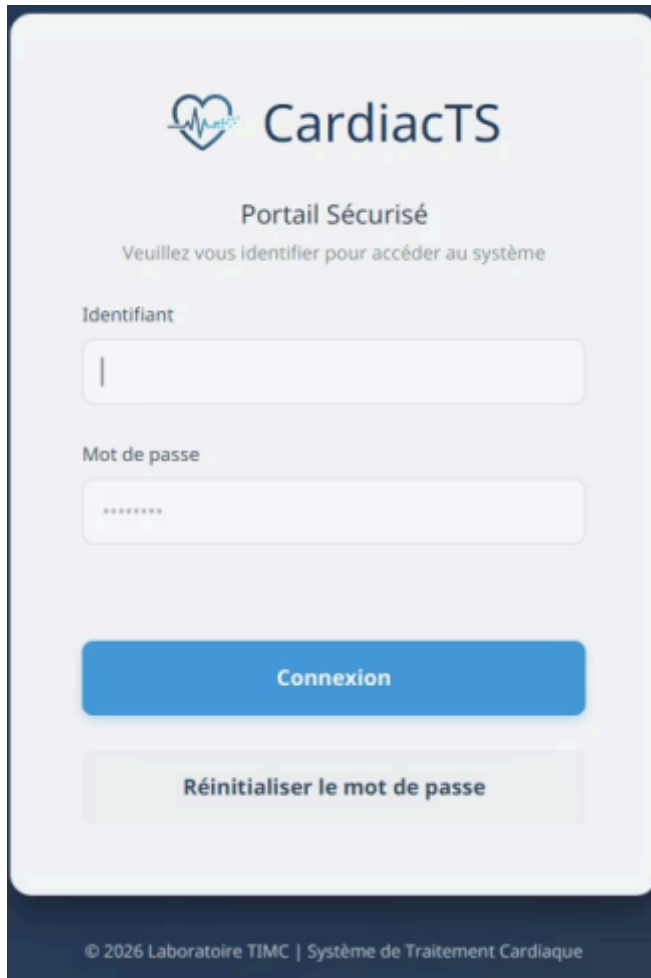
- **Entrée / Action** : Un administrateur ouvre l'onglet "**Génération de rapports d'audit**" après qu'un utilisateur a ajouté un cliché médical.
- **Résultat attendu** : Le tableau de bord affiche clairement :
"L'utilisateur Jean a uploadé l'imagerie ID#402 dans le dossier patient X, le 15 juin à 14h32".

Conclusions

L'intégration de la sécurité dans **CardiacTS** m'a fait réaliser qu'un système d'information de santé ne peut exister sans une confiance absolue dans ses données. J'ai mis en pratique le principe de "moindre privilège" grâce à l'architecture **RBAC**, et j'ai compris comment la traçabilité (**Audit**) protège autant l'organisation que l'utilisateur. Le masquage des secrets via **Dotenv** a également été une bonne pratique essentielle pour notre travail collaboratif sur **GitHub**.

Annexes

- **Annexe 1** : Capture de la page de connexion "**Portail Sécurisé – CardiacTS**".



The screenshot shows a web interface for 'CardiacTS'. At the top, there is a logo consisting of a heart with a pulse line and the text 'CardiacTS'. Below the logo, the text 'Portail Sécurisé' is displayed, followed by the instruction 'Veuillez vous identifier pour accéder au système'. There are two input fields: 'Identifiant' (username) and 'Mot de passe' (password). The password field is masked with dots. Below the input fields, there is a blue button labeled 'Connexion' and a light green button labeled 'Réinitialiser le mot de passe'. At the bottom of the page, there is a footer that reads '© 2026 Laboratoire TIMC | Système de Traitement Cardiaque'.

- **Annexe 2 :** *Extrait de la base de données (PostgreSQL) montrant les mots de passe hachés et la table des logs d'audit.*

Compétences associées

N°	Compétences associées
B1.1.1	Gérer les droits d'accès et les habilitations (Architecture RBAC)
B3.5.3b	Protéger l'identité numérique d'une organisation (Hachage jBCrypt)
B3.5.6a	Mettre en œuvre des mesures de sécurité et de traçabilité (Audit PostgreSQL)
B1.1.2	Gérer le code source en toute sécurité (Utilisation de fichiers .env / Git)